# Experiment Overview 2026-07-26

# Experiment Overview: TDD Workflows × Models × Prompt Styles

As of: 2026-07-26. Data basis: `experiments/runs/` (1127 runs total). This snapshot reports the **19 generic research questions with 855 runs**; the internal workflow-development line (13 RQs, 272 runs) is deliberately omitted.

**Author:** Marco Emrich (codecentric AG) — co-initiator of EXACT Coding together with Ferdinand Ade.

**Repository:** github.com/marcoemrich/agentic_coding_lab — all scripts, workflow definitions, run artifacts and the stylesheet are publicly versioned there.

## About the Study

This lab is the empirical validation platform for **EXACT Coding** (EXample-guided AI-Collaborative Test-driven Coding), described in the book EXACT Coding. The workflow variants under study deliberately span a spectrum: from vibe-coding baselines without any test structure, through EXACT-conformant setups with an explicit test list and an enforced red-green-refactor cadence, to a delayed-refactor control that implements first, tests afterwards, and cleans up exactly once at the end. This spectrum is the actual measuring apparatus: it makes it possible to isolate the individual EXACT building blocks — example-driven spec, complete test list, test-first discipline, periodic refactoring — against one another, instead of pitting "with TDD" as an undifferentiated package against "without TDD". It is precisely this isolation that carries the central finding of this snapshot: that correctness and code quality are driven by *different* building blocks.

This snapshot is dated 2026-07-26. The lab comprises 1127 runs across 32 research questions in total; reported here are the **19 generic questions with 855 runs** — those that pit prompt style, model, workflow structure, context architecture and harness against each other. The 13 questions of the internal workflow-development line (272 runs), which test individual prompt building blocks against one another, are deliberately omitted: they are tool development on the measuring apparatus and barely readable for an external audience without knowledge of the workflow version history. They are available in full in the repository under `research/workflow-dev/`.

The research front has shifted compared to earlier snapshots. Whether a structured approach measurably beats an unstructured one has been answered and replicated several times. Other questions are open today: the effective timing of the refactor lever (continuously in every cycle versus a single pass over the entire source at the end) and above all transferability across agent harnesses and model providers — the most recent additions to the lab are three further harnesses alongside Claude Code and a broad field of non-Anthropic models.

### Scope

The scope spans three axes. (1) **Harness** — four headless agent CLIs: Claude Code (`2.1.170`), OpenCode (`1.15.10`), pi (`0.81.1`) and cursor-cli (`cursor-agent`). All run **without a human in the loop**: the agent receives a prompt and works through to the end or to the budget timeout, without a human correcting, questioning or aborting in between. (2) **Models** — Anthropic (Opus 4.6/4.7/4.8/5, Sonnet 4.6/5, Haiku 4.5, Fable 5, each with and without thinking) plus third-party models reachable via Requesty (GLM 5.1/5.2, DeepSeek V4 Flash/Pro, Kimi K2.6/K2.7, GPT-5.6 Sol/Terra, Gemini 3.5 Flash, Mistral Medium 3.5, MiniMax M2.7/M3, Qwen3-235B, Grok 4.5, Composer 2.5). (3) **Target language** — exclusively **TypeScript** with an identical pnpm/tsx/Vitest/ESLint+SonarJS stack per run.

The findings hold **for** this stack. Transfer to other target languages (Python, Go, Java) is open and not examined here — in particular the complexity metrics via SonarJS/ESLint are bound to the TypeScript ecosystem. Equally open is transfer to **interactive HITL setups**: all findings describe what an agent produces unsupervised. Precisely the failure modes that cost correctness here (premature self-termination, incomplete test lists, wrongly guessed interface contracts) would be caught in an interactive setup by a single human question. The numbers are therefore to be read as a statement about autonomy robustness, not as an upper bound on what is achievable with the same tools under human guidance. The katas are synthetic and small (~30–320 Production LoC); web apps, database code and asynchronous systems are not covered.

### AI Disclosure

This snapshot was produced with the `/build-overview` skill in **Claude Code** (Anthropic Opus 5). Data-driven sections — the RQ overview table, coverage values, per-RQ finding lists, reproducibility and files tables — are generated deterministically from `research/{questions-*,workflow-dev}/*/{README,findings}.md` via `experiments/generate-snapshot-skeleton.py`. Synthesis sections (intro, per-RQ paragraphs, cross-RQ synthesis, limitations) are LLM-drafted and human-curated. The generation is therefore fully traceable.

## Key Findings

Four central findings from the 19 research questions — details and evidence in §4, cross-RQ synthesis in §5. In brief: the two core EXACT Coding building blocks work, but on **separate targets** — example mapping and a test-writing phase carry correctness, the enforced refactor cadence carries code quality. Together they yield the workflow that stays robust across both task types:

1. **EXACT Coding works — the combination of example mapping and tests-against-spec measurably beats vibe coding.** On the novel kata (claim-office), Correctness (external, `verification_pct`) falls from ≥ 0.96 to 0.28 as soon as work proceeds by vibe coding without a test-writing phase; example mapping as a spec style raises it by a further +48–76 percentage points over prose. Both correctness levers are the *specification* (concrete I/O examples) and the *formulation as tests against the spec* — not the red-green-refactor cycle itself (the naive "use TDD" run also reaches 1.00 correctness). Practical consequence: on novel domains, concrete I/O examples plus a test-writing phase are the most valuable correctness investment.
2. **Strict TDD measurably improves code quality.** On claim-office, a workflow with a periodic, isolated refactor step lowers the complexity peak to ~⅓ and the Smell Total to ~1/10 of vibe coding (`cognitive_max` 5.7 vs. 11–12, `smell_total` 1.3 vs. 12–16). The lever is the structured refactor discipline in

cadence, not the “TDD” label: the naive approach — an agent that only hears “use TDD” and is left to itself without an enforced red-green-refactor cadence — produces the heaviest code in the entire matrix ( `cognitive_max` 19.8), worse than no TDD at all. Practical consequence: for long-lived code a workflow with an enforced cleanup step per cycle pays off; a mere “do it in TDD” instruction does not.

3. **Example mapping is the dominant correctness lever on novel tasks — user story ≈ prose.** On claim-office, example mapping raises `verification_pct` by +48–76 percentage points over prose (Opus 4.7: 0.21 → 0.97; Opus 4.6 no-thinking: 0.23 → 0.87; Sonnet 4.6 no-thinking: 0.23 → 0.71), because concrete input/output examples resolve the domain ambiguities. User story performs practically identically to prose (Δ ≤ 8 pp). On training-known katas the effect is nil, and the lever is model-gated: Haiku 4.5 stays at 0 % in every style. Practical consequence: when writing a spec for a novel domain, concrete I/O pairs are the most valuable investment.
4. **A hybrid workflow with skill-based red/green in shared context and an isolated refactor subagent is the robust code-quality default — but no workflow wins universally.** This architecture is the only one that lands in the top 2 across both code-quality katas ( `cognitive_max` 5.7 on claim-office against 19.8 for minimal TDD, 6.5 on game-of-life against 21.8) and at the same time the only one with a 0 % outlier rate across ten replicates. At the same time, phase-isolated and hybrid workflows swap places depending on the model — the same workflow reaches 0.93 on one model and 0.67 on the next — and the fully phase-isolated variant drops from first place on the known kata to eighth on the novel one. Practical consequence: a refactor phase plus refactor isolation are the levers — but the best workflow has to be validated on your own model/task combination.

---

## 1. Research Questions Overview

### Research Questions (Claude Code)

| Ch. | RQ | Question | Status | Cells | Coverage | n Runs |
|---|---|---|---|---|---|---|
| 1.1 | RQ-prompt-correctness | Does example mapping raise correctness over prose and user story — and is the effect model-dependent? | active | 24 | 24/24 (100 %) | 129 |
| 1.2 | RQ-prompt-known-kata | Does prompt style (prose/user-story/example-mapping) affect correctness and code quality on a training-known kata (Game of Life) — and is this effect model-dependent? | active | 9 | 9/9 (100 %) | 45 |
| 2.1 | RQ-model-quality | How strongly do the available models (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8, Fable 5 — each with/without thinking) differ in code quality on a training-known kata under the strongest workflow? | active | 12 | 12/12 (100 %) | 44 |
| 2.2 | RQ-model-novel | How do Fable 5, Opus 4.8, Opus 4.7 and Opus 4.6 (each no-thinking) differ in correctness and code quality on a novel kata with ambiguities that differentiates more strongly than the training-known game-of-life? | active | 5 | 5/5 (100 %) | 30 |
| 3.1 | RQ-workflow-model | Does the quality of a TDD workflow depend on the model — is there a universally best workflow, or do different workflows swap places depending on the model? | active | 6 | 6/6 (100 %) | 49 |

| Ch. | RQ | Question | Status | Cells | Coverage | n Runs |
|---|---|---|---|---|---|---|
| 4.1 | RQ-tdd-quality | How does workflow structure (from oneshot through iterative to strict TDD with subagents) affect code quality, and does TDD strictness make a difference? | active | 16 | 16/16 (100 %) | 103 |
| 4.2 | RQ-tdd-correctness | Does external correctness (verification_pct) differ between TDD workflow variants on the novel claim-office kata? | active | 7 | 7/7 (100 %) | 36 |
| 4.3 | RQ-context | Which form of context structuring — isolated subagent contexts per TDD phase (v4.1), a shared, accumulated single context (v5.1), a hybrid with skill-based red/green in shared context and an isolated refactor subagent (v6.1), or a hybrid with isolated green and refactor subagents with shared-context test list/red (v7.1) — leads to better code quality? | active | 4 | 4/4 (100 %) | 21 |
| 4.4 | RQ-pocock-vs-v62 | How does the external Matt Pocock TDD skill (v9-pocock-tdd: single skill, inline phases, tail refactor) perform on claim-office-example-mapping against the internal default baseline v6.2-with-why-cleaned (multi-command + refactor subagent, per-cycle refactor) — on correctness, code quality, TDD discipline and cost? | active | 2 | 2/2 (100 %) | 11 |
| 5.1 | RQ-stability | How stable are code quality and TDD discipline per workflow across replicates, and under which conditions is n=3 a sufficient replicate count? | active | 6 | 5/6 (83 %) | 59 |

## Research Questions (OpenCode)

| Ch. | RQ | Question | Status | Cells | Coverage | n Runs |
|---|---|---|---|---|---|---|
| 1.1 | RQ-model-quality-oc | How do five models reachable via the OpenCode harness (Opus 4.7 via Portkey + four non-Anthropic models from the Portkey catalog) differ in code quality and TDD discipline on game-of-life-example-mapping with the v5.1-testlist-scope-fix-oc workflow? | active | 6 | 6/6 (100 %) | 30 |
| 1.2 | RQ-model-novel-oc | How do five models reachable via the OpenCode harness differ in correctness and TDD discipline on claim-office-example-mapping with the v5.1-testlist-scope-fix-oc workflow? | active | 8 | 8/8 (100 %) | 40 |

## Research Questions (pi)

| Ch. | RQ | Question | Status | Cells | Coverage | n Runs |
|---|---|---|---|---|---|---|
| 1.1 | RQ-model-quality-pi | How do the models reachable via the pi harness (Requesty routing) differ in code quality and TDD discipline on game-of-life-example-mapping with the v6.2.1-phase-continuation-pi workflow? | active | 10 | 10/10 (100 %) | 50 |
| 1.2 | RQ-model-novel-pi | How do the models reachable via the pi harness (Requesty routing) differ in correctness and TDD discipline on claim-office-example-mapping with the v6.2-with-why-cleaned-pi workflow? | active | 15 | 15/15 (100 %) | 75 |

## Research Questions (Cursor CLI)

| Ch. | RQ | Question | Status | Cells | Coverage | n Runs |
|---|---|---|---|---|---|---|
| 1.1 | RQ-model-quality-cursor | How do the models reachable via the cursor-cli harness (Opus, Composer, Grok) differ in code quality and TDD discipline on game-of-life-example-mapping? | active | 3 | 3/3 (100 %) | 15 |

### Research Questions (cross-harness)

| Ch. | RQ | Question | Status | Cells | Coverage | n Runs |
|---|---|---|---|---|---|---|
| 1.1 | RQ-harness | How does switching harness (Claude Code vs OpenCode vs pi) affect correctness, code quality and TDD discipline when model, workflow intention and prompt style are held constant? | active | 6 | 6/6 (100 %) | 38 |
| 1.2 | RQ-harness-requesty | How does switching harness (Claude Code vs OpenCode vs pi) affect correctness, code quality, TDD discipline and cost when model (opus-4-8 via Requesty), workflow intention and prompt style are held constant? | active | 8 | 8/8 (100 %) | 40 |
| 1.3 | RQ-cost-sol-pi-vs-opus-cc | How much cheaper is the GPT model gpt-5-6-sol on the pi harness compared to opus-4-8 on Claude Code — at identical prompt style and outcome-equivalent TDD workflow, across both katas? | active | 4 | 4/4 (100 %) | 20 |
| 1.4 | RQ-model-quality-cc-vs-pi | Does the code quality profile of Opus (opus-4-8) differ between the Claude Code and the pi harness, each with and without thinking, at constant workflow generation (v6.2)? | active | 4 | 4/4 (100 %) | 20 |

---

## 2. Experiment Design

### 2.1 Variables

**Workflow** — six generations (details: `research/workflow-dev/workflow-construction.md` — inventory):

| Workflow | Structure | TDD strictness |
|---|---|---|
| v1-oneshot | “Implement X.” | none |
| v2-iterative | “Plan step by step, then implement.” | none |
| v3-basic-tdd | Inline TDD, no skill/subagent (self-reporting) | minimal |
| v4-exact-subagents | Dedicated subagent per phase (predictor + red/green/refactor), fresh context | strict, multi-context |
| v4.1-testlist-scope-fix | v4 with test-list scope patch | strict, multi-context |
| v5-exact-single-context | All phases in one conversation, same phase script | strict, single-context |
| v5.1-testlist-scope-fix | v5 with test-list scope patch (aligned with v4.1) | strict, single-context |
| v6-hybrid | Hybrid: inline TDD + only refactor as subagent | strict, hybrid |
| v6.1-hybrid-testlist-scope-fix | v6-hybrid with test-list scope patch (current default basis) | strict, hybrid |
| v6.1-no-pep | v6.1 without pep talks (RQ-pep replication) | strict, hybrid |
| v7-hybrid-green-refactor | Like v6, but green *and* refactor as subagent | strict, more isolation |
| v7.1-hybrid-green-refactor-testlist-scope-fix | v7 with test-list scope patch | strict, more isolation |
| v8a-delayed-refactor-agent | Oneshot → tests added afterwards → single end-refactor agent ( `refactor.md` from v6.5.4) | delayed-refactor |
| v8b-delayed-refactor-native | Like v8a, but native inline refactor in v3 style, no agent | delayed-refactor |

Configuration: `experiments/workflows/<variant>/.claude/agents/` and `.claude/rules/`. Archived variants (v5.1-minimized, v6.2–v6.6, v6.5.x audits) are located under `experiments/workflows/_archive/`.

**Workflow mechanics in detail.** The six generations are not merely a scale of "more/less TDD", but a systematic variation of the EXACT Coding building blocks (test list, red, green, refactor) and their context architecture:

- **v1-oneshot / v2-iterative — vibe-coding baselines (no TDD).** A single agent reads the requirements and writes code in one step (v1) or with an explicit plan/checklist (v2); tests are only added afterwards on the basis of the example mapping. Serves as the yardstick for the value of TDD itself (see `experiments/workflows/v1-oneshot/.claude/rules/experiment-mode.md`).
- **v3-basic-tdd — minimal TDD without structure.** A single agent with the minimal instruction "use TDD" — no phase prompts, no subagents. Claude decides for itself how to structure the TDD process. Measures how far a bare instruction carries (`v3-basic-tdd/.claude/rules/experiment-mode.md`).
- **v4-exact-subagents / v4.1-testlist-scope-fix — strict TDD, multi-context.** Each TDD phase runs as a specialized subagent in an **isolated context** (`Task(subagent_type: "red")` etc.): `test-list` → `red` → `green` → `refactor`. Hypothesis: isolated contexts enforce discipline, but can lose state between phases. v4.1 adds the obligation "Cover every spec example" in the `test-list` subagent — closing the dominant failure mode on novel katas (incomplete test list) on Opus 4.7.
- **v5-exact-single-context / v5.1-testlist-scope-fix — strict TDD, single-context.** Identical phase script as v4, but all phases run in the **same context** as skill calls (`Skill(skill: "red")` etc.) instead of as subagents. Hypothesis: shared context preserves state, but can lead to loss of discipline. v5.1 mirrors v4.1 with the identical test-list scope patch.
- **v6-hybrid / v6.1-hybrid-testlist-scope-fix — hybrid with isolated refactor.** Red and green run inline as skills in the shared context (like v5), refactor runs as an isolated subagent (like v4). Hypothesis: combines the spec coherence of the single context with the discipline sharpening of subagent isolation at the most critical point (refactor). v6.1 is the current default basis and champion across several RQs. `v6.1-no-pep` tests the reduction of psychological rationales in red/green.
- **v7-hybrid-green-refactor / v7.1-…-testlist-scope-fix — hybrid with isolated green + refactor.** In addition to the refactor isolation from v6, green also runs as an isolated subagent. Test list and red remain in the shared context. Tests whether more isolation is equally better (Pareto-dominated by v6 on game-of-life: saves tokens, loses quality and correctness).
- **v8a-delayed-refactor-agent / v8b-delayed-refactor-native — delayed-refactor control.** Three sequential phases without TDD cycles: (1) oneshot implementation, (2) tests added afterwards against `prompt.md` with a coverage obligation, (3) a single end refactor. v8a uses the `refactor.md` subagent from v6.5.4 (APP + naming + mandatory attempt), v8b a native inline refactor in v3 style without an agent. Serves as the control axis for the hypothesis "periodic TDD refactor beats end refactor after vibe coding".

A deeper discussion of the mechanics, an inventory of the active v6.1 reduction line and the supporting RQ findings are in `research/workflow-dev/workflow-construction.md`. Which markers drive the parsing of the TDD metrics is documented in `experiments/workflows/MARKERS.md`. The archived v6.5.x line is located in `experiments/workflows/_archive/` and `research/_archive/workflow-dev-v1/`.

**Model × thinking** (lab variant IDs from `MODEL_CONFIGS` in `experiments/docker/run-batch.sh`):

| Lab variant ID | API ID | Thinking | Routing |
|---|---|---|---|
| `opus-4-7` | `claude-opus-4-7` | Adaptive | Direct |
| `opus-4-7-no-thinking` | `claude-opus-4-7` | off | Direct |
| `sonnet-4-6` | `claude-sonnet-4-6` | Extended | Direct |
| `sonnet-4-6-no-thinking` | `claude-sonnet-4-6` | off | Direct |
| `haiku-4-5` | `claude-haiku-4-5-20251001` | Extended | Direct |
| `haiku-4-5-no-thinking` | `claude-haiku-4-5-20251001` | off | Direct |
| `opus-4-7-portkey` | `@vertex-eu-global/anthropic.claude-opus-4-7` | Adaptive | Portkey |
| `opus-4-7-portkey-no-thinking` | `@vertex-eu-global/anthropic.claude-opus-4-7` | off | Portkey |
| `opus-4-6-portkey` | `@vertex-ai/anthropic.claude-opus-4-6` | Adaptive | Portkey |
| `opus-4-6-portkey-no-thinking` | `@vertex-ai/anthropic.claude-opus-4-6` | off | Portkey |
| `sonnet-4-6-portkey` | `@vertex-ai/anthropic.claude-sonnet-4-6` | Extended | Portkey |
| `sonnet-4-6-portkey-no-thinking` | `@vertex-ai/anthropic.claude-sonnet-4-6` | off | Portkey |
| `haiku-4-5-portkey` | `@vertex-ai/anthropic.claude-haiku-4-5@20251001` | Extended | Portkey |
| `haiku-4-5-portkey-no-thinking` | `@vertex-ai/anthropic.claude-haiku-4-5@20251001` | off | Portkey |

Direct and Portkey routings of the same model are separate variants and are counted as a shared cell only via an explicit `controls.model: {any: [...]}` clause per RQ.

**Kata × prompt style** (active katas in `experiments/katas/`):

| Kata base | Prompt styles | Verification suite | Note |
|---|---|---|---|
| game-of-life | prose, example-mapping, user-story | no | Code quality, large (~40 LoC), vitest-based |
| game-of-life-cli | prose, example-mapping, user-story | yes | CLI variant with external acceptance suite |
| mars-rover | prose, example-mapping, user-story | no | medium (~30 LoC), vitest-based |
| claim-office | prose, example-mapping, user-story | yes | Correctness, novel insurance domain (HPSMV/MHPCO), 15 scenarios |

| Kata base | Prompt styles | Verification suite | Note |
|---|---|---|---|
| claim-office-lite | prose, example-mapping, user-story | yes | Reduced claim-office variant (10 scenarios) for code quality research |

Prompt styles: - **prose**: description of the rules in prose, no test examples. - **example-mapping**: rule + 1–3 concrete input/output examples per rule. - **user-story**: “As X I want Y, so that Z” — description without examples.

### 2.2 Workflow → prompt mapping

For methodological symmetry (see top-level `README.md`, section ‘Methodology constraints’):

| Workflow | allowed prompt styles | Rationale |
|---|---|---|
| v1, v2 | prose only | Test examples in example-mapping would be a hidden test gift for non-TDD workflows → unfair towards the TDD workflows. |
| v3, v4(.1), v5(.1), v6(.1), v7(.1), v8a/b | all three | Examples serve as natural test cases — for TDD/refactor workflows this is the ideal form of the task. |

---

## 3. Methodology

The pipeline description was checked against `experiments/docker/Dockerfile`, `experiments/analyze-run.sh` and `experiments/aggregate-by-query.py` and corrected in two points compared to older snapshots: the container now pins **claude-code 2.1.170** (not 2.1.107) and additionally contains the three further harnesses (OpenCode, pi, cursor-agent). Step 4 of the pipeline is correspondingly harness-dependent — `claude --print` applies only to the Claude Code arm; OpenCode, pi and cursor-cli are started with their respective headless invocations. Analysis steps 5–6 are identical for all harnesses, because `analyze_transcript.py` has a dedicated transcript parser per harness and writes into the same `metrics.json` structure.

### 3.1 Run pipeline

1. Container image `docker-batch` (Node 22 slim, claude-code 2.1.170 / opencode 1.15.10 / pi 0.81.1 / cursor-agent pinned) is started.
2. Run dir `runs/<timestamp>_<kata>_<workflow>_<model>/` is created; workflow config (`.claude/agents/`, `.claude/rules/`) and kata prompt (`prompt.md`) are copied into it.
3. pnpm workspace with TypeScript, Vitest, ESLint+SonarJS is set up.
4. The respective harness runs headless, without HITL (Claude Code via `claude --print "$(< prompt.md)"`; OpenCode, pi and cursor-agent via their corresponding headless invocations).
5. `analyze-run.sh` writes `metrics.json` and `analysis-report.md`.
6. `aggregate-by-query.py <RQ>/` builds `runs.csv` and `summary.md` per RQ.

### 3.2 Metrics collected

Binding terms (column “Term”) are defined in the top-level `README.md` — alternative synonyms are forbidden because they collide or are ambiguous. Full metrics table including external references (Stryker, SonarJS, McCabe paper etc.) in the README section “Metrics”.

**Correctness**

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `tests_passing` | Correctness (internal) | Boolean: do the Vitest tests written by the agent run green at the end of the run? | `true` = better |
| `verification_pct` | Correctness (external) | Share of passed verification scenarios from an external acceptance suite that the agent never gets to see (0.0–1.0). Only for CLI katas with a `<basename>-verification/` directory. | higher = better |

**Efficiency**

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `duration_seconds` | — | Wallclock seconds of the `claude --print` run including all subagent spawns | lower = better |
| `total_tokens` | — | Sum of all tokens (input + output + cache) across all subagent spawns | lower = better |
| `context_utilization_pct` | — | Final context window utilization in the main context, in percent | informative |

**Code Mass & Size**

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `code_mass` | Code Mass (APP) | Weighted sum of the production code constructs (constants, invocations, conditionals, loops, assignments — graded weights by complexity) according to the *Absolute Priority Premise* (Micah Martin). Compares implementations more objectively than raw LoC. | lower = better |
| `cc_loc` | Production LoC | Production LoC excluding tests, from the clean-code reporter | lower = better (at equal correctness) |
| `test_lines` | Test LoC | Number of lines of test code (Vitest) | informative |
| `tests_total` | — | Number of tests written by the agent | informative |

**Code Quality (ESLint + SonarJS)**

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `cc_longest_function` | Complexity Peak | Longest function in lines — proxy for the worst spot in the code | lower = better |
| `cc_avg_loc_per_function` | — | Mean function size in lines | lower = better |
| `cc_median_loc_per_function` | — | Median function size (robust against individual long outliers) | lower = better |
| `cc_functions` | — | Number of functions | informative |
| `mccabe_max` / `mccabe_avg` / `mccabe_high_count` | — | McCabe cyclomatic complexity per function: maximum, mean, count above threshold. Classic branching metric. | lower = better |
| `cognitive_max` / `cognitive_avg` / `cognitive_high_count` | — | SonarSource cognitive complexity per function: weights nesting and control-flow breaks more strongly than McCabe, closer to humanly perceived complexity. Diagnostically the load-bearing main metric of this study. | lower = better |
| `smell_total` | Smell Total | Aggregated number of ESLint+SonarJS violations across all rules | lower = better |
| `smell_complexity` | — | Subset of `smell_total`: cognitive-complexity, max-depth, max-lines-per-function, max-params, no-nested-switch | lower = better |
| `smell_magic_numbers` | — | Subset: ESLint `no-magic-numbers` violations | lower = better |
| `smell_duplication` | — | Subset: SonarJS `no-duplicate-string` and related duplication rules | lower = better |
| `smell_code_quality` | — | Subset: SonarJS `no-collapsible-if`, `no-redundant-jump` etc., plus ESLint `no-unreachable` | lower = better |
| `coverage_statements_pct` | — | Statement coverage of the tests written by the agent (in %) | higher = better |
| `coverage_branches_pct` | — | Branch coverage of the tests written by the agent (in %) | higher = better |

**Test Strength**

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `mutation_score` | Mutation Score | Share of Stryker mutants killed by the agent's test suite (0.0–1.0): `(Killed + Timeout) / (Killed + Survived + Timeout + NoCoverage)`. Hidden metric — appears in no workflow prompt, hence Goodhart-resistant. Opt-in per RQ, only for `tests_passing = true`. | higher = better |

**TDD Discipline** (from `transcript.jsonl` + `transcript-subagents/`; driven by four markers in `experiments/workflows/MARKERS.md` — if a marker is missing, the corresponding metric silently falls to zero)

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `cycle_count` | — | Number of red-green-refactor cycles per run | informative (higher = more finely decomposed) |
| `refactorings_applied` | — | Number of explicitly applied refactoring steps | higher = better (for TDD workflows) |
| `predictions_correct` / `predictions_total` | — | Red-phase predictions about compile/runtime failure: correct vs. total. Depth of the agent's code understanding. 1–2 predictions per cycle depending on the workflow. | rate higher = better |

| Metric | Term | What it measures | Direction |
|---|---|---|---|
| `tests_passed_immediately` | — | Number of tests already green in the red phase — indicator of over-implementation in preceding green phases | lower = better |
| `avg_red_seconds` / `avg_green_seconds` / `avg_refactor_seconds` | — | Mean phase duration per cycle | informative |

### 3.3 Evaluation principles

- **Correctness first**: a run with `tests_passing=false` does not count as an equivalent solution.
- **Aggregate per kata**: workflow×model tables are built exclusively per kata.
- **Effect threshold**: at n=1 per cell, only differences with a factor ≥ 2 or clearly separated σ bands count as robust.

---

## 4. Results

### Research Questions (Claude Code)

#### 1.1 RQ-prompt-correctness — Does example mapping raise correctness over prose and user story — and is the effect model-dependent?

*Data basis: 129 runs · Coverage: 24/24 cells (100 %) at min_replicates=5.*

**Correctness (external) by model × prompt style × thinking** (higher = better; 🏆 = best style per row):

| Model | Mode | prose | example-mapping | user-story |
|---|---|---|---|---|
| opus-4-7 | +thinking | 0.29 | **0.95** 🏆 | 0.21 |
| opus-4-7 | −thinking | 0.21 | **0.97** 🏆 | 0.13 |
| opus-4-6 | +thinking | 0.24 | **0.72** 🏆 | 0.22 |
| opus-4-6 | −thinking | 0.23 | **0.87** 🏆 | 0.18 |
| sonnet-4-6 | +thinking | 0.21 | **0.35** 🏆 | — |
| sonnet-4-6 | −thinking | 0.23 | **0.71** 🏆 | 0.17 |
| haiku-4-5 | +thinking | 0.00 | 0.00 | 0.01 |
| haiku-4-5 | −thinking | 0.00 | 0.00 | 0.00 |

Values: mean(`verification_pct`), n=5 each (opus-4-6 EM n=4; opus-4-7 −thinking EM n=9). Haiku rows without a winner — all values ~0.

**Findings**:

- **F-prompt-correctness.1** — Weak models fail regardless of prompt style
- **F-prompt-correctness.2** — Example mapping raises correctness massively
- **F-prompt-correctness.3** — Thinking hurts with example mapping (Sonnet > Opus)
- **F-prompt-correctness.4** — User story ≈ prose, no measurable effect on correctness
- **F-prompt-correctness.5** — Spread under example mapping is model-dependent

On the novel kata, example mapping is the dominant correctness lever: +76 pp for Opus 4.7, +64 pp for Opus 4.6, +48 pp for Sonnet 4.6 without thinking. User story stays within 8 pp of prose across all models — the stakeholder perspective simply supplies no information about the domain rules. The lever is model-gated: Haiku 4.5 reaches 0.00 in all six cells. Surprisingly, thinking acts *negatively* under example mapping and inversely to model strength (Sonnet −36 pp, Opus 4.6 −15 pp, Opus 4.7 −2 pp); transcript analysis shows that weaker models question the semantics of the examples and construct alternative readings. Caveat: one kata, one workflow. findings.md

#### 1.2 RQ-prompt-known-kata — Does prompt style (prose/user-story/example-mapping) affect correctness and code quality on a training-known kata (Game of Life) — and is this effect model-dependent?

*Data basis: 45 runs · Coverage: 9/9 cells (100 %) at min_replicates=5.*

**`verification_pct` by prompt style × model** (higher = better; 🏆 = best style per row, all ties):

| Model | prose | user-story | example-mapping |
|---|---|---|---|
| opus-4-6-portkey-no-thinking | **1.00** 🏆 (σ=0) | **1.00** 🏆 (σ=0) | **1.00** 🏆 (σ=0) |
| sonnet-4-6-portkey-no-thinking | **1.00** 🏆 (σ=0) | **1.00** 🏆 (σ=0) | **1.00** 🏆 (σ=0) |
| haiku-4-5-portkey-no-thinking | 0.24 (σ=0.43) | 0.00 (σ=0) | **0.63** 🏆 (σ=0.51) |

**Findings**:

- **F-prompt-known-kata.1** — Opus and Sonnet deliver perfect correctness regardless of style
- **F-prompt-known-kata.2** — Haiku fails for capacity reasons, not style reasons
- **F-prompt-known-kata.3** — H1 confirmed: prompt style does not differentiate on strong models
- **F-prompt-known-kata.4** — H4 confirmed: the ambiguity mechanism does not apply on a training-known kata

- **F-prompt-known-kata.5** — H2 cannot be evaluated: code quality is only comparable across working runs
- **F-prompt-known-kata.6** — RQ-prompt-correctness prediction confirmed: prompt style does not differentiate on a training-known kata
- **F-prompt-known-kata.7** — The verification adapter eliminates interface artefacts

The control test for the previous RQ: on the training-known kata the prompt-style effect vanishes entirely for strong models — Opus and Sonnet deliver 30/30 runs at `verification_pct` = 1.00, with a spread between styles of exactly 0 pp. That settles the mechanism: example mapping works by resolving ambiguities, and the Conway rules have none. On Haiku it works through a different channel — not disambiguation but activation: in 5/5 user-story runs the model aborts immediately (12–17 s, no code), while with examples 4/5 runs work the task through. Methodologically important: the verification adapter, which imports the agent function directly instead of checking against a CLI contract, eliminated interface artefacts that had previously shown up as correctness errors. findings.md

### 2.1 RQ-model-quality — How strongly do the available models (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8, Fable 5 — each with/without thinking) differ in code quality on a training-known kata under the strongest workflow?

*Data basis: 44 runs · Coverage: 12/12 cells (100 %) at min_replicates=3.*

**Code quality by model** (means; lower = better except `verification_pct`; quality trophies are correctness-gated):

| Model | code_mass | smell_total | mccabe_max | cognitive_max | cc_longest_function | verification_pct | n |
|---|---|---|---|---|---|---|---|
| opus-5 | 172.67 | 1.67 | 3.00 | 2.00 | 6.33 | **1.00** 🏆 | 3 |
| opus-5-no-thinking | 149.33 | **1.67** 🏆 | 2.67 | 1.67 | 5.33 | **1.00** 🏆 | 3 |
| fable-5 | 163.00 | 3.00 | **2.00** 🏆 | **1.00** 🏆 | 8.33 | **1.00** 🏆 | 3 |
| fable-5-no-thinking | 163.33 | 2.33 | 2.67 | 1.67 | 6.67 | **1.00** 🏆 | 3 |
| opus-4-8 | **145.33** 🏆 | 2.67 | 4.33 | 5.33 | **4.33** 🏆 | **1.00** 🏆 | 3 |
| opus-4-8-no-thinking | 190.50 | 3.00 | 4.25 | 4.75 | 11.50 | **1.00** 🏆 | 4 |
| opus-4-7 | 159.00 | 2.33 | 3.33 | 3.00 | 7.00 | **1.00** 🏆 | 3 |
| opus-4-7-no-thinking | 166.60 | 2.60 | 4.50 | 4.40 | 8.10 | **1.00** 🏆 | 10 |
| opus-4-6-portkey | 173.00 | 4.33 | 6.67 | 12.00 | 19.33 | **1.00** 🏆 | 3 |
| opus-4-6-portkey-no-thinking | 175.67 | 4.33 | 7.67 | 13.00 | 18.67 | **1.00** 🏆 | 3 |
| sonnet-4-6 | 178.00 | 5.67 | 6.33 | 11.00 | 21.67 | **1.00** 🏆 | 3 |
| sonnet-4-6-no-thinking | 166.67 | 3.33 | 6.00 | 5.00 | 15.00 | 0.73 | 3 |

**Findings**:

- **F-model-quality.1** — Correctness (internal + external) on v4 is near-perfect independently of the model
- **F-model-quality.2** — Model ranking: Fable 5 and Opus 5 lead on complexity, Opus 4.8 on Code Mass; all three clearly ahead of Opus 4.6 and Sonnet
- **F-model-quality.3** — Thinking does not act uniformly; strong on code size for Opus 4.8, neutral for Opus 4.6, negative on cognitive_max for Sonnet
- **F-model-quality.4** — Token costs: Fable 5 and Sonnet/Opus 4.7 the cheapest, Opus 4.8 the most expensive; wallclock uniform
- **F-model-quality.5** — Contract conformance under an explicit API contract almost fully achieved; one Sonnet outlier redefines `Cell` as an object

With correctness saturated (eleven of twelve cells at 1.00), code quality alone separates the models — and it does so into three complementary profiles rather than one ranking. Fable 5 and Opus 5 hold the Complexity Peak close to the theoretical minimum (`cognitive_max` 1.0 and 2.0 respectively), while Opus 4.8 instead minimises Code Mass (APP) and the longest function (145.3 / 4.3). The gap to the preceding generation is large: on `cognitive_max`, a factor of ~12 separates Fable 5 from Opus 4.6. Thinking does not act uniformly — strongly on code size for Opus 4.8 (−45 `code_mass`), but clearly harmful for Sonnet (`cognitive_max` doubles to 11.0). Caveat: n=3 in most cells, one kata, one workflow. findings.md

### 2.2 RQ-model-novel — How do Fable 5, Opus 4.8, Opus 4.7 and Opus 4.6 (each no-thinking) differ in correctness and code quality on a novel kata with ambiguities that differentiates more strongly than the training-known game-of-life?

*Data basis: 30 runs · Coverage: 5/5 cells (100 %) at min_replicates=5.*

**Correctness (external) primary, code quality and cost secondary** (values in parentheses = leading, but without full correctness coverage):

| Model | n | verification_pct ↑ | σ | cognitive_max ↓ | mccabe_max ↓ | smell_total ↓ | total_tokens ↓ | duration_s ↓ |
|---|---|---|---|---|---|---|---|---|
| fable-5-no-thinking | 5 | 0.83 | 0.10 | (4.0) | (4.2) | (0.2) | **13.4 M** 🏆 | 7826 |
| opus-5-no-thinking | 5 | 0.88 | 0.11 | **2.8** 🏆 | 3.8 | **0.2** 🏆 | 24.6 M | 5931 |
| opus-4-8-no-thinking | 5 | **0.92** 🏆 | 0.09 | 7.4 | **7.0** 🏆 | 1.2 | 31.0 M | 5264 |
| opus-4-7-no-thinking | 10 | 0.67 | 0.36 | 10.5 | 7.9 | 1.8 | 13.7 M | **3693** 🏆 |

| Model | n | verification_pct ↑ | σ | cognitive_max ↓ | mccabe_max ↓ | smell_total ↓ | total_tokens ↓ | duration_s ↓ |
|---|---|---|---|---|---|---|---|---|
| opus-4-6-portkey-no-thinking | 5 | **0.93** 🏆 | 0.08 | 22.2 | 10.6 | 5.6 | 15.1 M | 4416 |

**Findings**:

- **F-model-novel.1** — opus-4-8 and opus-4-6 solve claim-office reliably, opus-5 and fable-5 land mid-field, opus-4-7 does not
- **F-model-novel.2** — The workflow × model interaction is the dominant effect
- **F-model-novel.3** — Correctness differentiates more strongly than code quality
- **F-model-novel.4** — A more precise mechanism on opus-4-7: test-list completeness, not subagent isolation
- **F-model-novel.5** — opus-4-8 buys the best code quality with ~2× the token cost
- **F-model-novel.6** — fable-5: the cleanest, most thoroughly tested code — but never the full spec

On the novel kata it is not code quality that separates the models but correctness itself — and the naive expectation “newer = better” does not hold: the two edges of the Opus 4 series lead (4.6 at 0.93, 4.8 at 0.92), while the middle 4.7 falls off bimodally at 0.67. The mechanism behind it is precisely located: not subagent isolation but an incomplete test list that omits entire spec operations — a single additional obligation (“Cover every spec example”) lifts 4.7 to 0.96 with a drastically narrower spread. Fable 5 shows the mirror image of Opus 4.8: the cleanest and most thoroughly tested code in the RQ, but never the full spec (max 14/15). Caveat: n=5, routing difference between 4.6 (Portkey) and the native models. findings.md

### 3.1 RQ-workflow-model — Does the quality of a TDD workflow depend on the model — is there a universally best workflow, or do different workflows swap places depending on the model?

*Data basis: 49 runs · Coverage: 6/6 cells (100 %) at min_replicates=5.*

`verification_pct` **by workflow × model** (higher = better; 🏆 per model column — the winner changes with the model):

| Workflow | opus-4-7 (n) | opus-4-6 (n) |
|---|---|---|
| v4-exact-subagents | 0.67 (10) | **0.93** (5) 🏆 |
| v5-exact-single-context | 0.87 (10) | 0.87 (5) |
| v6-hybrid | **1.00** (5) 🏆 | 0.68 (15) |

**Findings**:

- **F-workflow-model.1** — v4 and v6 swap places depending on the model
- **F-workflow-model.2** — Mechanism: orchestration delegation vs. explicit subagent prompt

The shortest RQ with the most inconvenient statement: there is no universally best workflow. v4 and v6 swap places depending on the model — v6 is optimal on Opus 4.7 at 1.00 and unstable on 4.6 at 0.68, v4 exactly the other way round (0.93 / 0.67). Only v5 is constant independently of the model (0.87 in both columns). The mechanism lies in the responsibility for orchestration: v6 delegates it to the model (skill invocation in shared context), which 4.7 handles, while 4.6 loses half the spec in roughly 40 % of runs — it implements only the quote part and ignores claim entirely, with internal tests still green. Consequence: workflow recommendations without model context do not generalise. findings.md

### 4.1 RQ-tdd-quality — How does workflow structure (from oneshot through iterative to strict TDD with subagents) affect code quality, and does TDD strictness make a difference?

*Data basis: 103 runs · Coverage: 16/16 cells (100 %) at min_replicates=5.*

**Code quality per workflow** (all metrics lower = better; 🏆 = best value per column. Never averaged across katas).

Kata `game-of-life`:

| Workflow | n | cognitive_max | mccabe_max | cc_longest_function | smell_total | cc_loc | code_mass |
|---|---|---|---|---|---|---|---|
| v1-oneshot | 10 | 18.8 | 12.8 | 31.7 | 4.8 | 33.6 | 155.0 |
| v2-iterative | 10 | 16.2 | 11.6 | 32.1 | 4.1 | 32.5 | 157.8 |
| v3-basic-tdd | 10 | 21.8 | 13.7 | 32.5 | 6.0 | 31.9 | 165.6 |
| v4.1-testlist-scope-fix | 5 | **6.4** 🏆 | **5.0** 🏆 | 16.4 | **2.4** 🏆 | 32.0 | 156.6 |
| v5.1-testlist-scope-fix | 5 | 17.6 | 10.2 | 20.8 | 4.8 | **26.6** 🏆 | 154.0 |
| v6.1-hybrid-… | 10 | 6.5 | 5.2 | **14.2** 🏆 | **2.4** 🏆 | 29.2 | 153.7 |
| v8a-delayed-refactor-agent | 5 | 10.6 | 7.4 | 17.6 | 3.0 | 31.2 | **142.0** 🏆 |
| v8b-delayed-refactor-native | 5 | 9.0 | 6.8 | 17.6 | **2.4** 🏆 | 31.0 | 145.8 |

Kata `claim-office`:

| Workflow | n | cognitive_max | mccabe_max | cc_longest_function | smell_total | cc_loc | code_mass |
|---|---|---|---|---|---|---|---|
| v1-oneshot | 5 | 12.2 | 8.4 | 40.4 | 11.6 | 269.4 | 835.4 |
| v2-iterative | 5 | 11.4 | 8.4 | 41.4 | 15.8 | 268.6 | 851.0 |

| Workflow | n | cognitive_max | mccabe_max | cc_longest_function | smell_total | cc_loc | code_mass |
|---|---|---|---|---|---|---|---|
| v3-basic-tdd | 5 | 19.8 | 15.4 | 51.6 | 16.8 | 317.4 | 992.4 |
| v4.1-testlist-scope-fix | 5 | 26.8 ⚠️ | 16.0 ⚠️ | 40.8 | 13.2 | **156.8** 🏆 | **621.6** 🏆 |
| v5.1-testlist-scope-fix | 6 | 14.8 | 10.2 | 32.7 | 6.8 | 167.2 | 692.7 |
| v6.1-hybrid-… | 7 | **5.7** 🏆 | **5.7** 🏆 | **18.1** 🏆 | **1.3** 🏆 | 191.1 | 861.3 |
| v8a-delayed-refactor-agent | 5 | 7.4 | 6.6 | 28.4 | 4.0 | 245.6 | 813.8 |
| v8b-delayed-refactor-native | 5 | 11.0 | 8.0 | 35.8 | 6.2 | 238.8 | 780.2 |

⚠️ v4.1-claim-office is bimodal (`cognitive_max` σ=24, max=68). Correctness: on game-of-life all eight workflows at `verification_pct` = 1.00; on claim-office 0.28 (v1/v2) to 1.00 (v3, v5.1, v6.1, v8a).

**Findings**:

- **F-tdd-quality.1** — Strict phase-structured workflows with a refactor phase drastically lower the complexity peaks
- **F-tdd-quality.2** — Naive "use TDD" (v3) yields no complexity advantage over non-TDD (v1/v2) on game-of-life
- **F-tdd-quality.3** — Single context (v5.1) loses the complexity advantage of phase-isolated subagents (v4.1) — but only on game-of-life
- **F-tdd-quality.4** — Correctness is workflow-dependent on the novel kata; v1/v2 vibe coding collapses on claim-office
- **F-tdd-quality.5** — The cost range between workflows spans an order of magnitude; strict workflows are 5–50× more expensive; kata complexity scales linearly
- **F-tdd-quality.6** — Vibe + end refactoring reaches the volume level of the strict TDD workflows at non-TDD cost; branching complexity stays weaker
- **F-tdd-quality.7** — The subagent mechanism for the end refactor beats the slash command on the large kata; level on the small kata
- **F-tdd-quality.8** — A test-writing phase rescues correctness on the novel kata; pure vibe coding fails
- **F-tdd-quality.9** — The v6.1 hybrid is the most robust TDD workflow across both katas; v4.1 is kata-unstable

The load-bearing RQ of the snapshot, and it separates two levers that tend to be sold as one package. **Correctness** hangs solely on the existence of a test phase: v1/v2 without tests collapse to 0.28 on claim-office, every workflow with a test phase lands at ≥ 0.96 — including v8a/v8b, which test afterwards. **Code quality**, by contrast, hangs exclusively on the enforced refactor cadence: v3 ("use TDD" without structure) produces the worst code in the matrix at `cognitive_max` 19.8, worse than vibe coding (11.4), while v6.1 reaches 5.7. The TDD label alone therefore carries nothing. The price: v6.1 costs 16× more tokens than v8a for an improvement from 7.4 to 5.7. Caveat: one model (opus-4-7-no-thinking), n=5 per claim-office cell. findings.md

### 4.2 RQ-tdd-correctness — Does external correctness (verification_pct) differ between TDD workflow variants on the novel claim-office kata?

*Data basis: 36 runs · Coverage: 7/7 cells (100 %) at min_replicates=3.*

**Correctness per workflow** (higher = better; 🏆 = best value per column, all ties):

| Workflow | n | verification_pct (mean ± std) | verification_passed / 15 (min – max) | tests_passing |
|---|---|---|---|---|
| v3-basic-tdd | 5 | **1.00 ± 0** 🏆 | 15 – 15 | **100 %** 🏆 |
| v4.1-testlist-scope-fix | 5 | 0.96 ± 0.09 | 12 – 15 | **100 %** 🏆 |
| v5.1-testlist-scope-fix | 6 | **1.00 ± 0** 🏆 | 15 – 15 | **100 %** 🏆 |
| v6.1-hybrid-… | 3 | **1.00 ± 0** 🏆 | 15 – 15 | **100 %** 🏆 |
| v7.1-hybrid-green-refactor-… | 3 | 0.98 ± 0.04 | 14 – 15 | **100 %** 🏆 |

**Findings**:

- **F-tdd-correctness.1** — Three of five TDD workflows solve claim-office perfectly; v4.1 and v7.1 occasionally lose scenarios
- **F-tdd-correctness.2** — v4.1 only reaches correctness through drastically higher effort per cycle
- **F-tdd-correctness.3** — The predictions-rate comparison is distorted by an unequal prediction base
- **F-tdd-correctness.4** — The wallclock range is 10×, the token range 9×; no correlation with correctness

Within the TDD family, correctness is no longer a scarce good: three of five variants solve claim-office completely in every run, the other two lose one scenario each. The dividing line is striking — both workflows with an *isolated* green subagent (v4.1, v7.1) carry one outlier each, all three with green in shared context are perfect; an isolated green does not see the test-list discussion and misses edge cases. The effort behind it, by contrast, spreads by a factor of 9–10: v4.1 runs 44.6 cycles and 54 minutes per run for the same correctness that v3 reaches with 3.8 cycles in 5 minutes. On this kata, structured workflows therefore justify themselves not through correctness but through code quality. findings.md

### 4.3 RQ-context — Which form of context structuring — isolated subagent contexts per TDD phase (v4.1), a shared, accumulated single context (v5.1), a hybrid with skill-based red/green in shared context and an isolated refactor subagent (v6.1), or a hybrid with isolated green and refactor subagents with shared-context test list/red (v7.1) — leads to better code quality?

*Data basis: 21 runs · Coverage: 4/4 cells (100 %) at min_replicates=3.*

**Code quality, correctness and cost by context architecture** (🏆 = best value per column; `verification_pct` higher = better, all others lower = better):

| Workflow | n | cognitive_max | mccabe_max | cc_longest_function | smell_total | code_mass | cc_loc | verification_pct | duration_seconds | total_tokens |
|---|---|---|---|---|---|---|---|---|---|---|
| v4.1 (all isolated) | 5 | 26.8 ± 24.1 | 16.0 ± 9.0 | 40.8 ± 27.1 | 13.2 ± 7.5 | **621.6 ± 65.6** 🏆 | **156.8 ± 38.0** 🏆 | 0.96 ± 0.09 | 3 229 ± 920 | **14.10 M ± 2.99** 🏆 |
| v5.1 (all shared) | 6 | 14.8 ± 4.2 | 10.2 ± 2.6 | 32.7 ± 10.2 | 6.8 ± 7.6 | 692.7 ± 78.8 | 167.2 ± 27.9 | **1.00 ± 0** 🏆 | **641 ± 122** 🏆 | 18.73 M ± 5.35 |
| v6.1 (refactor isolated) | 3 | **4.3 ± 1.5** 🏆 | **5.0 ± 1.7** 🏆 | **16.7 ± 6.7** 🏆 | **1.3 ± 1.2** 🏆 | 920.7 ± 55.2 | 184.3 ± 4.9 | **1.00 ± 0** 🏆 | 1 424 ± 781 | 30.16 M ± 18.56 |
| v7.1 (green + refactor isolated) | 3 | **5.0 ± 1.0** 🏆 | **4.67 ± 0.58** 🏆 | **19.3 ± 2.5** 🏆 | **2.3 ± 2.3** 🏆 | 801 ± 3.6 | 187.3 ± 29.2 | 0.98 ± 0.04 | 1 970 ± 715 | 26.11 M ± 6.20 |

**Findings**:

- **F-context.1** — The refactor subagent delivers the complexity advantage; additional green isolation does not change the picture
- **F-context.2** — The refactor subagent distributes functionality across more building blocks; green isolation dampens the more-code effect
- **F-context.3** — Correctness does not distinguish the architectures
- **F-context.4** — Four very different cost profiles
- **F-context.5** — Two hybrid positions with similar code quality and different cost profiles

The four-point comparison locates the quality lever precisely: it arises from the **isolated refactor subagent** and from nothing else. v6.1 and v7.1 share exactly this element and reach practically identical complexity peaks ( `cognitive_max` 4.3 / 5.0, all differences within σ); the additional green isolation in v7.1 brings no further gain. When *all* phases run isolated (v4.1), by contrast, it hurts — the subagents reconstruct the overall architecture again and again across 44.6 cycles and accumulate complexity that no phase sees as a whole (σ `cognitive_max` = 24.1). Notable: v6.1 writes the most code at 920 LOC and still has the fewest smells — cleanliness comes from the structure, not from frugality. Caveat: n=3 in the hybrid cells. findings.md

### 4.4 RQ-pocock-vs-v62 — How does the external Matt Pocock TDD skill (v9-pocock-tdd: single skill, inline phases, tail refactor) perform on claim-office-example-mapping against the internal default baseline v6.2-with-why-cleaned (multi-command + refactor subagent, per-cycle refactor) — on correctness, code quality, TDD discipline and cost?

*Data basis: 11 runs · Coverage: 2/2 cells (100 %) at min_replicates=3.*

**Attribution.** The external skill is vendored unmodified from mattpocock/skills (path `skills/engineering/tdd/`, MIT-licensed), retrieved **2026-05-26**. The measurements below therefore describe that specific snapshot; upstream has evolved since, so this is not a statement about the current version of that skill.

**External single-skill TDD against the internal multi-command baseline:**

| Axis | v6.2-with-why-cleaned (n=8) | v9-pocock-tdd (n=3) | Winner |
|---|---|---|---|
| **Correctness** `verification_pct` (higher = better) | 0.96 ± 0.09 | **1.00 ± 0** 🏆 | Pocock slightly |
| `tests_passing` rate | 100 % | 100 % | Tie 🏆🏆 |
| **Code quality** `cognitive_max` (lower = better) | **5.00 ± 1.77** 🏆 | 14.33 ± 1.53 | v6.2 |
| `mccabe_max` (lower = better) | **4.50 ± 0.76** 🏆 | 11.67 ± 0.58 | v6.2 |
| `cc_longest_function` (lower = better) | **12.38 ± 1.41** 🏆 | 32.33 ± 1.53 | v6.2 |
| `smell_total` (lower = better) | **0.38 ± 0.74** 🏆 | 6.67 ± 8.96 | v6.2 |
| `code_mass` (lower = better) | 878.5 ± 91 | **748.3 ± 62** 🏆 | Pocock |
| **Cost** `duration_seconds` (lower = better) | 2530 ± 401 | **570 ± 106** 🏆 | Pocock |
| `total_tokens` (lower = better) | 44.4 M ± 3.4 M | **13.1 M ± 4.6 M** 🏆 | Pocock |
| **Discipline** `refactorings_applied` | 24.88 ± 6.90 | 0 ± 0 | different by design |
| `cycle_count` | 37.38 ± 1.60 | 14.00 ± 3.46 | different by design |
| `tests_passed_immediately` (lower = stricter) | 15.12 ± 5.84 | **2.33 ± 4.04** 🏆 | Pocock |
| `predictions_correct_rate` (higher = better) | **97.2 %** 🏆 | 89.9 % | v6.2 |

**Findings**:

- **F-4.4.1** — Pocock and v6.2 are equally correct
- **F-4.4.2** — v6.2 produces cleaner code, Pocock more compact code
- **F-4.4.3** — Pocock is ~70–78 % cheaper
- **F-4.4.4** — The tail refactor does not trigger on claim-office
- **F-4.4.5** — Pocock takes fewer, larger steps
- **F-4.4.6** — Pocock skips less often

An externally developed TDD skill with inline phases and a tail refactor reaches the same correctness as the internal baseline (1.00 vs 0.96) at 70–78 % lower cost — and delivers code that is worse by factors ( `cognitive_max` 14.3 vs 5.0, `smell_total` 6.7 vs 0.4). The cause is cleanly identified and supports the refactor-cadence finding from §4.6: the tail formulation “After all tests pass, look for refactor candidates” triggered **zero** refactorings in 3/3 runs, while the per-

cycle variant runs 24.9. With green tests, the model rates the code as good enough absent additional prompt pressure. Caveat: n=3 on the Pocock side — but at > 3 σ the effect sizes are directionally stable. findings.md

### 5.1 RQ-stability — How stable are code quality and TDD discipline per workflow across replicates, and under which conditions is n=3 a sufficient replicate count?

*Data basis: 59 runs · Coverage: 5/6 cells (83 %) at min_replicates=10.*

**Code quality by workflow at n=10** (lower = better; 🏆 = best value per column):

| Workflow | code_mass | smell_total | mccabe_max | cognitive_max | cc_longest_function | n |
|---|---|---|---|---|---|---|
| v1-oneshot (prose) | 155.00 | 4.80 | 12.80 | 18.80 | 31.70 | 10 |
| v2-iterative (prose) | 157.80 | 4.10 | 11.60 | 16.20 | 32.10 | 10 |
| v3-basic-tdd (EM) | 165.60 | 6.00 | 13.70 | 21.80 | 32.50 | 10 |
| v4-exact-subagents (EM) | 166.60 | 2.60 | **4.50** 🏆 | **4.40** 🏆 | **8.10** 🏆 | 10 |
| v5-exact-single-context (EM) | **152.60** 🏆 | 4.10 | 8.90 | 14.50 | 17.40 | 10 |
| v6-hybrid (EM) | 158.60 | **2.20** 🏆 | **4.50** 🏆 | 5.20 | 13.10 | 10 |

**Findings**:

- **F-stability.1** — The core finding from RQ-tdd-quality (v4 dominates code complexity, v3 comes last) replicates at n=10 with the same sign
- **F-stability.2** — Workflow stability is not uniform; v4 has a 10 % outlier rate despite a low typical value; v5 is the widest workflow
- **F-stability.3** — At n=3 the full workflow ranking is correct in only ~25–60 % of cases; v4 as "best" is more robust
- **F-stability.4** — At n=10, correctness stays at 100 % independently of model and workflow
- **F-stability.5** — Token consumption shows an extremely high spread for v4 and v5
- **F-stability.6** — TDD discipline forms workflow-characteristic bands
- **F-stability.7** — Test strength (`mutation_score`) has its own stability profile; v6-hybrid is the most stable workflow, v4 the least stable

The methodological backstop of the lab: at n=10 the ranking holds at the edges but not in the middle — "v4 is clearly better than everything else" is reliably detectable with n=3, whereas the complete five-workflow ordering is only correct in 16–63 % of subsamples. From this follows the lab rule: large effects may be reported with n=3, marginal differences need n≥10. The stability asymmetry matters: v4 has the best median but derails in 1 of 10 runs (`cognitive_max` = 17, the entire logic in a 28-line function), while v6 stays consistently predictable with a 0 % outlier rate and σ = 0.005 on `mutation_score`. The mean alone conceals this. Caveat: one model, one kata. findings.md

## Research Questions (OpenCode)

### 1.1 RQ-model-quality-oc — How do five models reachable via the OpenCode harness (Opus 4.7 via Portkey + four non-Anthropic models from the Portkey catalog) differ in code quality and TDD discipline on game-of-life-example-mapping with the v5.1-testlist-scope-fix-oc workflow?

*Data basis: 30 runs · Coverage: 6/6 cells (100 %) at min_replicates=5.*

**Code quality as the primary outcome, correctness as a gating precondition** (excerpt; trophies only for models with `verification_pct` = 1.0):

| Metric | Direction | opus-4-7-portkey | glm-5-1 | gemini-3-5-flash | kimi-k2-6 | deepseek-v4-flash | deepseek-v4-pro |
|---|---|---|---|---|---|---|---|
| `verification_pct` (mean) | higher | **1.00** 🏆 | **1.00** 🏆 | **1.00** 🏆 | 0.57 | **1.00** 🏆 | 0.85 |
| `smell_total` (mean) | lower | 3.6 | **2.8** 🏆 | 4.0 | 4.4 | 4.0 | 4.2 |
| `cognitive_max` (mean) | lower | **11.4** 🏆 | **11.6** 🏆 | 16.0 | 9.4 | 13.2 | 11.4 |
| `mccabe_max` (mean) | lower | 7.6 | **7.0** 🏆 | 10.4 | 7.6 | 9.4 | 8.6 |
| `cc_longest_function` (mean) | lower | **18.6** 🏆 | 19.8 | **18.6** 🏆 | 15.2 | 27.6 | 15.0 |
| `lines_of_code` (mean) | lower | **38.2** 🏆 | 46.4 | 52.2 | 22.4 | 44.8 | 24.6 |
| `duration_seconds` (mean) | lower | 231 | 835 | **153** 🏆 | 1083 | 612 | 2381 |
| `cost_usd` (mean, $/perfect-run) | lower | $1.84 | $0.74 | $1.06 | $2.65 | **$0.10** 🏆 | $0.46 |

**Findings**:

- **F-1.1** — Opus 4.7 writes the most compact implementation
- **F-1.2** — GLM 5.1 holds Opus-level complexity
- **F-1.3** — Kimi-K2 writes too few tests and fails external verification
- **F-1.4** — Gemini 3.5 Flash: fast, but the most complex code
- **F-1.5** — Skill-tool compliance is model-dependent
- **F-1.6** — DeepSeek-V4-Flash: the cheapest path to a correct solution
- **F-1.7** — DeepSeek-V4-Pro: skill-compliance champion, but tail risk in duration

A second harness makes non-Anthropic models comparable — and the price/quality field fans out widely. Opus 4.7 writes the most compact solution (38.2 Production LoC, median 3.3 LoC per function — it consistently extracts small helpers), GLM 5.1 holds Opus-level complexity at a third of the cost, and DeepSeek-V4-Flash reaches full correctness for ~$0.10 per run, an order of magnitude below Opus' $1.84. The most important methodological finding concerns marker compliance: Gemini Flash writes practically no prediction markers (0.4 per run) and is nevertheless fully correct — low marker counts measure format recognition, not TDD discipline. Kimi-K2 drops out of the trophy pool (0.57) because it minimises the test list. Caveat: n=5, one kata. findings.md

### 1.2 RQ-model-novel-oc — How do five models reachable via the OpenCode harness differ in correctness and TDD discipline on claim-office-example-mapping with the v5.1-testlist-scope-fix-oc workflow?

*Data basis: 40 runs · Coverage: 8/8 cells (100 %) at min_replicates=5.*

**Correctness (external) primary, code quality secondary** (excerpt; trophies only at `verification_pct` = 1.0):

| Metric | Direction | opus-4-7-portkey | glm-5-1 | mistral-medium-3-5 | kimi-k2-6 | gemini-3-5-flash | deepseek-v4-flash | deepseek-v4-pro | minimax-m2-7 |
|---|---|---|---|---|---|---|---|---|---|
| `verification_pct` (mean) | higher | **1.00** 🏆 | **1.00** 🏆 | 0.95 | 0.84 | 0.80 | 0.60 | 0.60 | 0.04 |
| `smell_total` (mean) | lower | **0.8** 🏆 | 4.0 | 23.6 | 20 | 18 | 13.4 | 16.6 | 10.2 |
| `cognitive_max` (mean) | lower | **9.8** 🏆 | 12.2 | 74.8 | 21.8 | 40.2 | 11.6 | 17.4 | 11.4 |
| `mccabe_max` (mean) | lower | **7.6** 🏆 | 9.2 | 33.6 | 17.6 | 23.4 | 9.2 | 11.0 | 7.6 |
| `cc_longest_function` (mean) | lower | **25.4** 🏆 | 28.8 | 120 | 54.4 | 98.4 | 31.6 | 42.2 | 30.0 |
| `cost_usd` (mean, $/run) | lower | $5.90 | **$2.10** 🏆 | $24.69 † | $2.78 | $2.23 | $0.28 ‡ | $0.11 ‡ | $2.40 |
| `duration_seconds` (mean) | lower | **664** 🏆 | 1726 | 4051 | 1811 | 395 | 1279 | 956 | 1428 |

† The Mistral cost is an OpenCode integration artefact (missing prompt caching), not a model finding. ‡ DeepSeek values include two CLI-contract aborts.

**Findings**:

- **F-1.1** — Opus 4.7 and GLM 5.1 reach full correctness; trade-off code quality ↔ cost
- **F-1.2** — Kimi K2.6 and Gemini 3.5 Flash: peak correctness with a variance tail
- **F-1.3** — MiniMax M2.7: a stable spec misunderstanding, not an isolated case
- **F-1.4** — Predictions format compliance is NOT predictive of correctness
- **F-1.5** — Code Mass spread within a model: Flash and MiniMax bimodal/wide
- **F-1.6** — Cost efficiency per perfect run: GLM 5.1 is deterministic AND cheap
- **F-1.7** — Mistral Medium 3.5: high correctness against high complexity and the highest cost
- **F-1.8** — DeepSeek V4 (flash + pro): the workflow-compatibility drop dominates over spec comprehension

On the novel kata the model field separates drastically: Opus 4.7 and GLM 5.1 solve all five replicates completely, MiniMax M2.7 fails in five out of five (0.04) — reproducibly, with 30.8 self-written, green tests. The model consistently builds a different spec than the verification suite expects; internal tests prove nothing here. GLM 5.1 is the practical winner on the cost axis: the same determinism guarantee as Opus at a third of the price ($2.10 vs $5.90), bought with slightly higher complexity. Two caveats cloud individual values: Mistral's extreme cost is a caching integration bug in the harness, and the DeepSeek values mix two failure modes (CLI contract breach in early runs vs. model performance). findings.md

## Research Questions (pi)

### 1.1 RQ-model-quality-pi — How do the models reachable via the pi harness (Requesty routing) differ in code quality and TDD discipline on game-of-life-example-mapping with the v6.2.1-phase-continuation-pi workflow?

*Data basis: 50 runs · Coverage: 10/10 cells (100 %) at min_replicates=5.*

**Code quality, lower = better** (trophies only for cells with `tests_passing` = 100 %):

| Model | smell_total | cognitive_max | mccabe_max | code_mass | cc_longest_function | tests_passing |
|---|---|---|---|---|---|---|
| glm-5-2 | **1.0** 🏆 | 7.8 | 6.6 | 178.2 | 22.6 | 100 % |
| sonnet-5 | 2.2 | **6.6** 🏆 | **5.0** 🏆 | 183.0 | 19.6 | 100 % |
| kimi-k2-7 | 3.0 | 10.8 | 7.2 | 150.4 | 21.6 | 100 % |
| opus-4-8 | 3.4 | 9.6 | 6.8 | 149.2 | 17.4 | 100 % |
| gpt-5-6-sol | 3.6 | 13.4 | 9.4 | **134.8** 🏆 | 21.2 | 100 % |
| deepseek-v4-pro | 4.0 | 14.0 | 10.2 | 158.4 | 25.4 | 100 % |
| minimax-m3 | 8.4 | **6.6** 🏆 | 5.2 | 212.2 | **15.0** 🏆 | 100 % |
| glm-5-1 | 2.2 | 7.2 | 6.0 | 144.8 | 22.2 | 80 % |
| gpt-5-6-terra | 6.0 | 7.8 | 6.0 | 136.4 | 23.2 | 80 % |
| qwen3-235b | 1.8 | 6.4 | 3.4 | 206.6 | 42.4 | 0 % |

**Findings**:

- **F-1.1** — glm-5-2 delivers the cleanest code, sonnet the lowest complexity
- **F-1.2** — deepseek and gpt-5-6-sol solve the kata correctly, but with high complexity
- **F-1.3** — Correctness clusters at the top, with qwen as a total failure
- **F-1.4** — TDD discipline varies strongly without correlating with correctness
- **F-1.5** — Costs spread by a factor of 6 at comparable quality

The third harness brings the widest model field (ten cells) and shows that the quality axes run partly orthogonally: glm-5-2 leads on Smell Total (1.0), sonnet-5 on both complexity measures (6.6 / 5.0) — no model dominates all three. Correctness clusters at the top (seven of ten cells at `verification_pct` = 1.00), with qwen3-235b as the clear floor: it builds code but never gets it green in any run — a genuine capability deficit, not an abort. Practically relevant is the cost gap: the lowest-smell models are also the most expensive (glm-5-2 ~$2.53, sonnet-5 ~$2.83 against kimi-k2-7 ~$0.60); a “cheap AND clean” model is missing from the field. Caveat: costs are list-price estimates, not billed amounts. findings.md

### 1.2 RQ-model-novel-pi — How do the models reachable via the pi harness (Requesty routing) differ in correctness and TDD discipline on claim-office-example-mapping with the v6.2-with-why-cleaned-pi workflow?

*Data basis: 75 runs · Coverage: 15/15 cells (100 %) at min_replicates=5.*

**Correctness (external), higher = better** (🏆 only for cells with `verification_pct` ≥ 0.99 at σ ≤ 0.03):

| Model | `verification_pct` mean | σ | `tests_passing` rate |
|---|---|---|---|
| opus-4-8-no-thinking | **1.00** 🏆 | 0.00 | 100 % |
| glm-5-2 | **1.00** 🏆 | 0.00 | 100 % |
| gpt-5-6-sol | **1.00** 🏆 | 0.00 | 100 % |
| kimi-k2-7 | **1.00** 🏆 | 0.00 | 100 % |
| opus-4-8 | **0.99** 🏆 | 0.03 | 100 % |
| sonnet-5-no-thinking | 0.84 | 0.15 | 100 % |
| deepseek-v4-pro-no-thinking | 0.80 | 0.45 | 80 % |
| minimax-m3-no-thinking | 0.77 | 0.44 | 80 % |
| kimi-k2-7-no-thinking | 0.73 | 0.42 | 80 % |
| sonnet-5 | 0.72 | 0.19 | 100 % |
| gpt-5-6-terra | 0.69 | 0.42 | 80 % |
| deepseek-v4-pro | 0.60 | 0.55 | 100 % |
| minimax-m3 | 0.20 | 0.45 | 100 % |
| qwen3-235b | 0.00 | 0.00 | 0 % |
| qwen3-235b-no-thinking | 0.00 | 0.00 | 0 % |

**Findings**:

- **F-1.1** — Correctness clusters dichotomously, with a graduated middle zone
- **F-1.2** — qwen3-235b builds code but never solves the kata
- **F-1.3** — TDD discipline and correctness do not correlate
- **F-1.4** — The reasoning switch does not shift correctness
- **F-1.5** — Perfect correctness at widely differing costs

With 15 cells, the broadest model survey in the lab, and it shows a three-part distribution: a perfect cluster (five cells at ≈ 1.00, σ ≤ 0.03), a total-failure cluster (qwen3-235b at 0.00 in both arms) and a wide, run-unstable middle (0.20–0.84 with σ up to 0.55) — there the same cell swings between 0 and 15 passed scenarios across replicates. Two findings with practical value: the reasoning switch does not shift correctness (even on Opus 4.8, where it demonstrably takes effect, ±0.01), and marker compliance says nothing about the outcome — gpt-5-6-sol solves the kata just as perfectly with 20.8 predictions as Opus does with 70. On cost, the perfect cells spread by a factor of 5.7 ($2.54 to $14.43). findings.md

## Research Questions (Cursor CLI)

### 1.1 RQ-model-quality-cursor — How do the models reachable via the cursor-cli harness (Opus, Composer, Grok) differ in code quality and TDD discipline on game-of-life-example-mapping?

*Data basis: 15 runs · Coverage: 3/3 cells (100 %) at min_replicates=5.*

**Code quality and correctness** (all quality metrics lower = better; all three cells 100 % correct, hence no correctness gating):

| Metric (direction) | opus-cursor | composer-cursor | grok-cursor |
|---|---|---|---|
| `cognitive_max` (↓) | 16.6 | **8.2** 🏆 | 10.6 |
| `cognitive_avg` (↓) | 15.3 | **5.93** 🏆 | 7.2 |
| `mccabe_max` (↓) | 10.6 | **7.6** 🏆 | 8.8 |
| `mccabe_avg` (↓) | 4.33 | **2.63** 🏆 | 3.38 |
| Smell Total `smell_total` (↓) | 4.0 | **3.4** 🏆 | 3.6 |
| Production LoC `lines_of_code` (↓) | **27.8** 🏆 | 59.2 | 42.8 |
| Code Mass (APP) `code_mass` (↓) | **141.8** 🏆 | 182.2 | 149.2 |
| Correctness (internal) `tests_passing` (↑) | **100 %** 🏆 | **100 %** 🏆 | **100 %** 🏆 |

| Metric (direction) | opus-cursor | composer-cursor | grok-cursor |
|---|---|---|---|
| `duration_seconds` (↓) | 198 | **120.8** 🏆 | 169 |
| `total_tokens` (↓) | 1.74 M | **768 k** 🏆 | 1.17 M |

**Findings**:

- **F-1.1** — Composer writes the least complex solution, Opus the most concise
- **F-1.2** — Model spread confirmed: the cursor-cli harness is able to discriminate
- **F-1.3** — Opus uses the TDD marker mechanics most densely

The newest harness in the lab, and it delivers the cleanest evidence that **conciseness and simplicity come apart**: Opus writes the shortest solution at 27.8 Production LoC and at the same time carries the highest cognitive complexity (16.6), while Composer writes more than twice as much code at 59.2 LoC with half the complexity (8.2). Code inspection confirms the mechanism: Opus packs the logic into one dense function with triply nested loops, Composer extracts named constants and helpers and separates the passes into flat individual steps. Anyone using “less code” as a quality proxy is measuring the opposite of maintainability here. Caveat: costs run through the Cursor subscription, hence no cost comparison with the other harnesses. findings.md

## Research Questions (cross-harness)

### 1.1 RQ-harness — How does switching the harness (Claude Code vs OpenCode vs pi) affect correctness, code quality and TDD discipline when model, workflow intention and prompt style are held constant?

*Data basis: 38 runs · Coverage: 6/6 cells (100 %) at min_replicates=5.*

**Pivot across six cells (kata × harness)** at constant model, workflow and prompt style:

| Outcome | Direction | CC × claim (n=8) | OC × claim (n=5) | pi × claim (n=5) | CC × GOL (n=10) | OC × GOL (n=5) | pi × GOL (n=5) |
|---|---|---|---|---|---|---|---|
| `verification_pct` (mean ± σ) | higher = better | 0.96 ± 0.09 | **1.00** 🏆 ± 0 | **1.00** 🏆 ± 0 | **1.00** 🏆 ± 0 | **1.00** 🏆 ± 0 | **1.00** 🏆 ± 0 |
| `input+output` (mean, k tokens, **without cache**) | lower = better | **161** 🏆 | 468 | 1971 | **36** 🏆 | 156 | 287 |
| `cost_usd` (mean, list price) | lower = better | $30.47 | $18.80 | **$11.20** 🏆 | $6.22 | $2.26 | **$1.65** 🏆 |
| `duration_seconds` (mean) | lower = better | 2530 ± 401 | 2230 ± 952 | **1647** 🏆 ± 205 | 627 ± 117 | 516 ± 196 | **317** 🏆 ± 43 |
| `code_mass` (APP, mean) | lower = better | 879 ± 91 | 827 ± 99 | **807** 🏆 ± 16 | 153 ± 14 | **149** 🏆 ± 12 | 158 ± 13 |
| `cognitive_max` (mean) | lower = better | 5.0 ± 1.8 | 4.8 ± 3.0 | **4.2** 🏆 ± 1.6 | **4.3** 🏆 ± 2.8 | 6.2 ± 2.6 | 7.6 ± 3.1 |
| `cc_longest_function` (mean) | lower = better | **12.4** 🏆 ± 1.4 | 15.0 ± 7.0 | 14.6 ± 1.7 | **12.2** 🏆 ± 6.9 | 17.0 ± 5.2 | 18.2 ± 5.3 |
| `refactorings_applied` (mean) | higher = better | **24.9** 🏆 ± 6.9 | 19.0 ± 11.4 | 16.8 ± 2.8 | **7.9** 🏆 ± 1.9 | 5.0 ± 2.8 | 3.0 ± 0.7 |

`total_tokens` is not directly comparable across the harnesses (different cache counting) — the fair efficiency proxy is `input + output`.

**Findings**:

- **F-harness.1** — Correctness is harness-invariant; CC × claim-office shows a slight spread
- **F-harness.2** — Token footprint and list-price cost: pi is the cheapest variant
- **F-harness.3** — Code Mass (APP) is harness-invariant; mccabe/longest/cognitive vary with the kata
- **F-harness.4** — Claude Code harness glitch: premature `end_turn` on claim-office (thinking variant)
- **F-harness.5** — TDD discipline is harness-invariant; refactor frequency falls monotonically CC → OC → pi
- **F-harness.6** — pi cycle inflation on claim-office: markedly more red markers than CC/OC at the same test count

At constant model and workflow, correctness is harness-invariant (five of six cells deterministically perfect) — the choice of agent tool therefore shifts cost and quality profile, not the quality of the result. Most instructive is the apparent contradiction on cost: on claim-office, pi pushes roughly twelve times as many fresh input tokens through the model as Claude Code (1971 k vs 161 k) and is nevertheless barely a third as expensive. The resolution: CC keeps the raw token count low by sending the same growing context through the cache again and again across ~37 cycles — 44 million cache reads, which even at the discounted tariff are more expensive than pi’s cache-free full-price bill. Caveat: frozen Portkey snapshot; the cost question is re-measured under Requesty in §4.6.2. findings.md

### 1.2 RQ-harness-requesty — How does switching the harness (Claude Code vs OpenCode vs pi) affect correctness, code quality, TDD discipline and cost when model (opus-4-8 via Requesty), workflow intention and prompt style are held constant?

*Data basis: 40 runs · Coverage: 8/8 cells (100 %) at min_replicates=5.*

**Four harnesses at constant model (opus-4-8 via Requesty), workflow and prompt style.**

Kata `claim-office` (external correctness counts):

| Metric (direction) | CC | OC | pi | cursor |
|---|---|---|---|---|
| `verification_pct` (higher) | 0.93 | 0.88 | 0.99 | **1.0** 🏆 |
| `tests_passing` rate (higher) | **100 %** 🏆 | **100 %** 🏆 | **100 %** 🏆 | **100 %** 🏆 |
| `cost_usd` $ (lower) | 32.89 | 22.30 | 14.43 | **9.22** 🏆 |

| Metric (direction) | CC | OC | pi | cursor |
|---|---|---|---|---|
| `total_tokens` (lower) | 49.9 M | 34.1 M | 13.8 M | **13.8 M** 🏆 |
| `duration_seconds` (lower) | 3149 | 2393 | 1884 | **1001** 🏆 |

Kata `game-of-life` (all cells `verification_pct` = 1.0):

| Metric (direction) | CC | OC | pi | cursor |
|---|---|---|---|---|
| `cost_usd` $ (lower) | 3.45 | 1.99 | 1.78 | **1.48** 🏆 |
| `total_tokens` (lower) | 4.09 M | 1.96 M | **1.07 M** 🏆 | 1.74 M |
| `cognitive_max` (lower) | **5.0** 🏆 | 12.6 | 11.0 | 16.6 |
| `mccabe_max` (lower) | **4.6** 🏆 | 8.8 | 8.0 | 10.6 |
| `smell_total` (lower) | **2.2** 🏆 | 3.2 | 3.4 | 4.0 |
| `refactorings_applied` (higher) | **8.8** 🏆 | 3.2 | 2.8 | 2.6 |

Cursor runs on `claude-opus-4-8-medium` (medium effort) — the quality deficit is an effort effect and a harness effect at once and cannot be separated.

**Findings**:

- **F-1.1** — Correctness is harness-invariant
- **F-1.2** — cursor is the cheapest and fastest harness; pi leads among CC/OC/pi
- **F-1.3** — Claude Code delivers the leanest Complexity Peak on game-of-life; cursor the highest
- **F-1.4** — TDD discipline is structurally the same across all harnesses, except for refactor intensity

Repeating the harness comparison under uniform cost measurement and with cursor as a fourth arm confirms the core finding: correctness is harness-invariant (0.88–1.00, all differences within replicate spread), cost spreads by a factor of 3.5 ($9.22 to $32.89) and wallclock by a factor of 3. The quality difference on game-of-life is clear and mechanistically explained: Claude Code holds `cognitive_max` at 5.0 against 11.0–16.6 for the others — and applies almost three times as many refactorings to do so, at 8.8. Cursor wins the cost axis partly through the tariff (native list price instead of the Requesty surcharge), not through efficiency alone. Caveat: cursor runs on a medium-effort model, so its quality deficit is confounded. findings.md

### 1.3 RQ-cost-sol-pi-vs-opus-cc — How much cheaper is the GPT model gpt-5-6-sol on the pi harness compared to opus-4-8 on Claude Code — at the same prompt style and with an outcome-equivalent TDD workflow, across both katas?

*Data basis: 20 runs · Coverage: 4/4 cells (100 %) at min_replicates=5.*

**Cost switching comparison of two practical bundles** (model AND harness vary together):

Kata `claim-office`:

| Metric (direction) | sol-pi | opus-cc |
|---|---|---|
| `cost_usd` $ (lower) | **2.54** 🏆 | 32.89 |
| `total_tokens` (lower) | **2.09 M** 🏆 | 49.9 M |
| `duration_seconds` (lower) | **503** 🏆 | 3149 |
| `verification_pct` (higher) | **1.00** 🏆 | 0.93 |
| `cognitive_max` (lower) | 9.2 | **3.0** 🏆 |
| `mccabe_max` (lower) | 6.8 | **3.8** 🏆 |
| Smell Total `smell_total` (lower) | 15.4 | **0.0** 🏆 |

Kata `game-of-life`:

| Metric (direction) | sol-pi | opus-cc |
|---|---|---|
| `cost_usd` $ (lower) | **1.09** 🏆 | 3.45 |
| `duration_seconds` (lower) | **240** 🏆 | 719 |
| `verification_pct` (higher) | **1.0** 🏆 | **1.0** 🏆 |
| `cognitive_max` (lower) | 13.4 | **5.0** 🏆 |
| Smell Total `smell_total` (lower) | 3.6 | **2.2** 🏆 |

**Findings**:

- **F-1.1** — sol-pi is drastically cheaper on both katas — ~13× on the expensive one
- **F-1.2** — The price advantage costs no correctness — on claim-office sol-pi is even more accurate
- **F-1.3** — Cheaper does not mean cleaner: sol-pi carries higher complexity and more smells throughout

The switching question in its purest form: moving from the expensive to the cheap bundle saves 68 % to 92 % depending on the kata and runs roughly three times faster — **without forfeiting correctness**; on the novel kata the cheap bundle is even more consistent at 1.00 (σ = 0) than the expensive one (0.93, σ = 0.12). The price is maintainability, and markedly so: on claim-office a Smell Total of 15.4 stands against 0.0 and `cognitive_max` 9.2 against 3.0. Rule of thumb: the cheap bundle for throughput-critical work with tolerable rework, the expensive one where low complexity justifies the surcharge. Binding caveat: model and harness vary together — the effect is their sum, not either one alone. findings.md

### 1.4 RQ-model-quality-cc-vs-pi — Does the code-quality profile of Opus (opus-4-8) differ between the Claude Code and the pi harness, each with and without thinking, at a constant workflow generation (v6.2)?

*Data basis: 20 runs · Coverage: 4/4 cells (100 %) at min_replicates=5.*

**Complexity and code quality** (all lower = better; all four cells 100 % correct):

| Metric (direction) | CC thinking | CC no-think | pi thinking | pi no-think |
|---|---|---|---|---|
| `cognitive_max` (↓) | **5.0** 🏆 | 5.6 | 9.6 | 8.2 |
| `cognitive_avg` (↓) | **3.17** 🏆 | 3.87 | 5.57 | 7.4 |
| `mccabe_avg` (↓) | 2.16 | **2.11** 🏆 | 2.9 | 3.13 |
| Smell Total `smell_total` (↓) | 2.2 | 2.0 | 3.4 | **1.2** 🏆 |
| Production LoC `lines_of_code` (↓) | 44.6 | 40.0 | **35.2** 🏆 | 42.2 |
| Code Mass (APP) `code_mass` (↓) | 158.6 | 171.8 | **149.2** 🏆 | 159.6 |
| Correctness (internal) `tests_passing` (↑) | **100 %** 🏆 | **100 %** 🏆 | **100 %** 🏆 | **100 %** 🏆 |
| `duration_seconds` (↓) | 718.6 | 579.2 | 339.4 | **318** 🏆 |

**Findings**:

- **F-1.1** — The harness effect on complexity dominates the thinking effect
- **F-1.2** — Thinking does not reliably lower code complexity on either harness
- **F-1.3** — pi is more concise and faster, CC less complex and more smell-stable

A two-factor test with a clear ordering: the harness path shapes the complexity profile more strongly than the reasoning level. Across both thinking levels, the same Opus writes consistently less complex code on one harness (`cognitive_max` ~5 against ~8–10), while the thinking effect *within* a harness stays smaller and inconsistent. Notable: thinking acts less on the mean than on the spread — without thinking, dense single-function outliers appear (one pi run puts the entire logic into a single function, `cognitive_avg` = 17 at 27 LoC), the same density pattern as with Opus on the cursor harness. Binding caveat: harness and workflow line cannot be separated here (two lines of the same generation). findings.md

---

## 5. Cross-RQ Synthesis

1. **The training-known kata measures almost nothing — and that is exactly what makes it valuable as a control.** On `game-of-life`, all three prompt styles in RQ-prompt-known-kata, all eight workflows in RQ-tdd-quality and eleven of twelve model cells in RQ-model-quality reach the same perfect correctness. Every factor that moves 70 percentage points on the novel kata is invisible there. Anyone evaluating only on known tasks systematically measures null effects and wrongly concludes the factors are equivalent. The consequence for your own evaluations: the benchmark must contain ambiguities, otherwise it is blind — and conversely, the known kata can be used deliberately to switch off correctness as a confounder and measure pure code quality.

2. **Internal tests are worthless as an acceptance criterion — across all model and harness RQs.** `tests_passing` sits at almost uniformly 100 % in RQ-model-quality, RQ-model-quality-oc, RQ-model-novel-oc, RQ-model-quality-pi and RQ-model-novel-pi, while the external suite spreads from 0.00 to 1.00 in those same cells. The patterns behind this differ and are all typical of agents: one model writes 30.8 green tests against a self-consistently misunderstood spec (RQ-model-novel-oc); another minimises the test list and tests only what it implemented anyway (RQ-model-quality-oc); a third ignores an entire spec operation whose tests it never created (RQ-model-novel). An agent that writes its own tests thereby also defines its own measure of success. In practice that means: acceptance needs criteria the agent has never seen.

3. **Every winner found is context-bound — across three independent axes.** The workflow ranking flips with the model (RQ-workflow-model: the same workflow at 0.93 on one model, 0.67 on the next), with the kata (RQ-tdd-quality: first place on one, eighth on the other) and with the harness (RQ-model-quality-cc-vs-pi: the same model carries less complex code on one path). None of the three axes can be predicted from the others. The finding is inconvenient because it limits the transferability of recommendations — but it is consistent across all affected RQs and the practical lesson is unambiguous: a workflow recommendation without a statement of model, task type and tool is not a recommendation.

4. **Whatever behaviour the prompt structure steers, it steers via the refactor invocation — not via the wording of diligence.** The comparison in RQ-pocock-vs-v62 shows it most sharply: an external TDD skill that phrases refactoring as an afterthought ("after the tests pass, look for candidates") triggers exactly zero refactorings in three out of three runs and lands at `cognitive_max` 14.3 — against 5.0 for the variant with a refactor phase per cycle. The same mechanic explains the harness difference in RQ-harness-requesty: the harness with the lowest Complexity Peak is the one that actually invokes the refactor step roughly three times as often. It is not the mention of quality in the prompt that produces quality, but the enforced execution of the step.

5. **The cost axis is largely decoupled from the outcome axis — and that is a practical opportunity.** In RQ-cost-sol-pi-vs-opus-cc, switching to a cheaper model-harness bundle saves 68–92 % of the cost at equal or better correctness; in RQ-harness-requesty, cost spreads by a factor of 3.5 across four harnesses without moving correctness; in RQ-model-quality-oc, the cheapest model solves the task for one eighteenth the price of the most expensive. What the surcharge buys is in all three cases the same and only the same: lower complexity and fewer smells. Anyone who needs a correct result and not the maintainability — scripts, glue code, throwaway prototypes — is currently paying a multiple as a matter of routine for an advantage they do not use.

---

## 6. Limitations

- **TypeScript only.** All runs use the same pnpm/tsx/Vitest/ESLint+SonarJS stack. The load-bearing quality metrics (`cognitive_max`, `mccabe_max`, `smell_total`) are bound to this tooling ecosystem; whether the workflow effects turn out the same in Python, Go or Java is untested.

- **Synthetic katas only, and only two carry the main load.** `game-of-life` (~30–40 Production LoC) and `claim-office` (~150–320) supply the bulk of the evidence. The kata axis is thereby reduced to “small and training-known” against “medium-sized and novel”. Existing codebases, web applications, database and async systems do not appear — precisely where refactoring is most expensive, data is missing.
- **Headless, without a human in the loop.** The numbers describe unsupervised autonomy. Several of the documented correctness losses are failure modes that a single human question would catch: premature self-termination, incomplete test lists, wrongly guessed CLI contracts. For interactive use the values are therefore a lower bound, not a prediction.
- **Small cells.** The standard is n=5, several cells at n=3. From the stability RQ it is known that at n=3 the complete workflow ranking is correctly reproduced in only 16–63 % of subsamples — large effects are robust, marginal differences are not. Where rankings with narrow margins appear in the text, they are to be read as provisional accordingly.
- **Costs are list-price estimates, not billed amounts.** All `cost_usd` values arise from measured tokens × public tariff. Discounts, smart routing and workspace-specific terms are absent. In addition the harnesses partly carry different tariff channels (native list price versus gateway surcharge) — part of the harness cost difference is a tariff effect, not an efficiency effect.
- **Routing confounding between model generations.** Some models ran via Portkey, some via Requesty, some natively. Where cells from different routing paths are compared, the effect is not cleanly separable from routing; the affected RQ sections flag this in each case.
- **One coverage gap.** All 19 reported research questions are fully populated (100 % of cells at the respective `min_replicates` threshold). The only exception lies in the workflow-development line omitted here.
- **The model comparison is a snapshot in time.** The model landscape moves faster than the data collection: several model generations appeared between the earliest and the most recent runs in this snapshot. Absolute model rankings age correspondingly fast; the structural findings (which factor drives which outcome) are considerably more stable than the names in the tables.

---

## 7. Reproducibility

All data and analysis scripts are in the repository:

- `research/questions-{claude,opencode,pi,cursor-cli,cross}/*/README.md` — RQ definitions (frontmatter with factors/controls/outcomes)
- `research/questions-{claude,opencode,pi,cursor-cli,cross}/*/findings.md` — persistent finding lists
- `research/workflow-dev/*/` — the workflow-development line omitted here (13 RQs, 272 runs), same structure
- `experiments/runs/*/metrics.json` — raw data per run
- `experiments/aggregate-by-query.py` — RQ-specific aggregation
- `experiments/batch-plan-from-rq.py` — batch plan generation from RQ frontmatter
- `experiments/docker/Dockerfile` + `run-batch.sh` + `batch.sh` — container pipeline
- `experiments/analyze-run.sh` + `analyze_transcript.py` — run analysis

Container pins: `claude-code@2.1.170`, `opencode-ai@1.15.10`, `@earendil-works/pi-coding-agent@0.81.1`, `pnpm@9.15.9` (see `experiments/docker/Dockerfile`).

---

## 8. Files

| Path | Content |
|---|---|
| `research/questions-claude/1.1-prompt-style-correctness/findings.md` | RQ-prompt-correctness — Does example mapping raise correctness over prose and user story — and is the effect model-dependent? |
| `research/questions-claude/1.1-prompt-style-correctness/runs.csv` | RQ-prompt-correctness aggregated run metrics |
| `research/questions-claude/1.2-prompt-style-known-kata/findings.md` | RQ-prompt-known-kata — Does prompt style (prose/user-story/example-mapping) affect correctness and code quality on a training-known kata (Game of Life) — and is this effect model-dependent? |
| `research/questions-claude/1.2-prompt-style-known-kata/runs.csv` | RQ-prompt-known-kata aggregated run metrics |
| `research/questions-claude/2.1-model-effect-code-quality/findings.md` | RQ-model-quality — How strongly do the available models (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8, Fable 5 — each with/without thinking) differ in code quality on a training-known kata under the strongest workflow? |
| `research/questions-claude/2.1-model-effect-code-quality/runs.csv` | RQ-model-quality aggregated run metrics |
| `research/questions-claude/2.2-model-effect-novel-kata/findings.md` | RQ-model-novel — How do Fable 5, Opus 4.8, Opus 4.7 and Opus 4.6 (each no-thinking) differ in correctness and code quality on a novel kata with ambiguities that differentiates more strongly than the training-known game-of-life? |
| `research/questions-claude/2.2-model-effect-novel-kata/runs.csv` | RQ-model-novel aggregated run metrics |
| `research/questions-claude/3.1-workflow-model-interaction/findings.md` | RQ-workflow-model — Does the quality of a TDD workflow depend on the model — is there a universally best workflow, or do different workflows swap places depending on the model? |
| `research/questions-claude/3.1-workflow-model-interaction/runs.csv` | RQ-workflow-model aggregated run metrics |
| `research/questions-claude/4.1-tdd-effect-code-quality/findings.md` | RQ-tdd-quality — How does workflow structure (from oneshot through iterative to strict TDD with subagents) affect code quality, and does TDD strictness make a difference? |
| `research/questions-claude/4.1-tdd-effect-code-quality/runs.csv` | RQ-tdd-quality aggregated run metrics |

| Path | Content |
|---|---|
| `research/questions-claude/4.2-tdd-effect-correctness/findings.md` | RQ-tdd-correctness — Does external correctness (verification_pct) differ between TDD workflow variants on the novel claim-office kata? |
| `research/questions-claude/4.2-tdd-effect-correctness/runs.csv` | RQ-tdd-correctness aggregated run metrics |
| `research/questions-claude/4.3-tdd-context-engineering/findings.md` | RQ-context — Which form of context structuring — isolated subagent contexts per TDD phase (v4.1), a shared, accumulated single context (v5.1), a hybrid with skill-based red/green in shared context and an isolated refactor subagent (v6.1), or a hybrid with isolated green and refactor subagents with shared-context test list/red (v7.1) — leads to better code quality? |
| `research/questions-claude/4.3-tdd-context-engineering/runs.csv` | RQ-context aggregated run metrics |
| `research/questions-claude/4.4-external-tdd-pocock-vs-v62/findings.md` | RQ-pocock-vs-v62 — How does the external Matt Pocock TDD skill (v9-pocock-tdd: single skill, inline phases, tail refactor) perform on claim-office-example-mapping against the internal default baseline v6.2-with-why-cleaned (multi-command + refactor subagent, per-cycle refactor) — on correctness, code quality, TDD discipline and cost? |
| `research/questions-claude/4.4-external-tdd-pocock-vs-v62/runs.csv` | RQ-pocock-vs-v62 aggregated run metrics |
| `research/questions-claude/5.1-workflow-stability/findings.md` | RQ-stability — How stable are code quality and TDD discipline per workflow across replicates, and under which conditions is n=3 a sufficient replicate count? |
| `research/questions-claude/5.1-workflow-stability/runs.csv` | RQ-stability aggregated run metrics |
| `research/questions-opencode/1.1-model-quality-oc/findings.md` | RQ-model-quality-oc — How do five models reachable via the OpenCode harness (Opus 4.7 via Portkey + four non-Anthropic models from the Portkey catalog) differ in code quality and TDD discipline on game-of-life-example-mapping with the v5.1-testlist-scope-fix-oc workflow? |
| `research/questions-opencode/1.1-model-quality-oc/runs.csv` | RQ-model-quality-oc aggregated run metrics |
| `research/questions-opencode/1.2-model-novel-kata-oc/findings.md` | RQ-model-novel-oc — How do five models reachable via the OpenCode harness differ in correctness and TDD discipline on claim-office-example-mapping with the v5.1-testlist-scope-fix-oc workflow? |
| `research/questions-opencode/1.2-model-novel-kata-oc/runs.csv` | RQ-model-novel-oc aggregated run metrics |
| `research/questions-pi/1.1-model-quality-pi/findings.md` | RQ-model-quality-pi — How do the models reachable via the pi harness (Requesty routing) differ in code quality and TDD discipline on game-of-life-example-mapping with the v6.2.1-phase-continuation-pi workflow? |
| `research/questions-pi/1.1-model-quality-pi/runs.csv` | RQ-model-quality-pi aggregated run metrics |
| `research/questions-pi/1.2-model-novel-kata-pi/findings.md` | RQ-model-novel-pi — How do the models reachable via the pi harness (Requesty routing) differ in correctness and TDD discipline on claim-office-example-mapping with the v6.2-with-why-cleaned-pi workflow? |
| `research/questions-pi/1.2-model-novel-kata-pi/runs.csv` | RQ-model-novel-pi aggregated run metrics |
| `research/questions-cursor-cli/1.1-model-quality-cursor/findings.md` | RQ-model-quality-cursor — How do the models reachable via the cursor-cli harness (Opus, Composer, Grok) differ in code quality and TDD discipline on game-of-life-example-mapping? |
| `research/questions-cursor-cli/1.1-model-quality-cursor/runs.csv` | RQ-model-quality-cursor aggregated run metrics |
| `research/questions-cross/1.1-harness-effect/findings.md` | RQ-harness — How does switching harness (Claude Code vs OpenCode vs pi) affect correctness, code quality and TDD discipline when model, workflow intention and prompt style are held constant? |
| `research/questions-cross/1.1-harness-effect/runs.csv` | RQ-harness aggregated run metrics |
| `research/questions-cross/1.2-harness-requesty/findings.md` | RQ-harness-requesty — How does switching harness (Claude Code vs OpenCode vs pi) affect correctness, code quality, TDD discipline and cost when model (opus-4-8 via Requesty), workflow intention and prompt style are held constant? |
| `research/questions-cross/1.2-harness-requesty/runs.csv` | RQ-harness-requesty aggregated run metrics |
| `research/questions-cross/1.3-cost-sol-pi-vs-opus-cc/findings.md` | RQ-cost-sol-pi-vs-opus-cc — How much cheaper is the GPT model gpt-5-6-sol on the pi harness compared to opus-4-8 on Claude Code — at identical prompt style and outcome-equivalent TDD workflow, across both katas? |
| `research/questions-cross/1.3-cost-sol-pi-vs-opus-cc/runs.csv` | RQ-cost-sol-pi-vs-opus-cc aggregated run metrics |
| `research/questions-cross/1.4-opus-cc-vs-pi/findings.md` | RQ-model-quality-cc-vs-pi — Does the code quality profile of Opus (opus-4-8) differ between the Claude Code and the pi harness, each with and without thinking, at constant workflow generation (v6.2)? |
| `research/questions-cross/1.4-opus-cc-vs-pi/runs.csv` | RQ-model-quality-cc-vs-pi aggregated run metrics |
| `experiments/runs/` | All run directories with source, transcript, metrics |